

Medical Cost Prediction Using Machine Learning: A Detailed Research Study with Implementation

GTU Data Science Mini Project - External Viva Submission

Author: Chaudhary Marmikkumar Ashvinbhai

Enrollment: 251370680002

Department: Data Science | Gujarat Technological University

Abstract

Rising healthcare expenditure creates financial pressure on insurers, hospitals, and patients. Accurate prediction of annual medical insurance charges supports fair premium pricing, budgeting, and risk assessment. This research presents an end-to-end machine learning system trained on the Medical Cost Personal Dataset (1,338 records, 7 attributes). After exploratory analysis, categorical encoding, and an 80/20 train-test split, a Random Forest Regressor achieved MAE of \$1,982.25, RMSE of \$2,517.21, and R^2 of 0.9331. Smoking status emerged as the dominant cost driver (mean charge \$32,050 for smokers vs \$8,434 for non-smokers). The project includes a Jupyter analytical notebook, bulk CSV prediction export, and a Streamlit deployment interface. This paper documents literature background, system design, complete Python implementation with code explanations, experimental results, and societal implications.

1. Introduction

Health insurance translates uncertain future medical expenses into predictable premium payments. Actuaries historically relied on demographic tables and manual rating factors. Modern electronic health and insurance records enable supervised learning models that estimate charges from structured attributes: age, sex, body mass index (BMI), dependents, smoking behavior, and US region.

The research problem is regression: predict continuous target variable charges (USD) from input vector X . Challenges include non-linear interactions (especially smoking), heterogeneous feature types, and the need for reproducible pipelines suitable for both offline analysis and online inference.

Contributions of this work: (1) comprehensive literature comparison of ML methods for insurance pricing; (2) documented preprocessing and modeling pipeline in scikit-learn; (3) interpretable EDA confirming domain expectations; (4) deployment-ready Streamlit application; (5) open project structure for GitHub portfolio submission as required by GTU mini-project guidelines.

2. Literature Review and Existing Methods

2.1 Statistical and Linear Models

Multiple Linear Regression (MLR) estimates charges as a linear combination of features. MLR offers interpretable coefficients but underfits when smoking creates a step-change in costs. Polynomial and ridge/lasso extensions reduce bias-variance trade-offs yet still struggle with sharp non-linearities.

2.2 Tree-Based Ensemble Learning

Decision trees recursively partition feature space. Random Forest (Breiman, 2001) aggregates predictions from many decorrelated trees, reducing overfitting. Gradient boosting (XGBoost, LightGBM) often wins Kaggle tabular competitions. For datasets near 1,000-2,000 rows, Random Forest provides strong accuracy with minimal tuning - motivating its selection in this project.

2.3 Neural and Kernel Methods

Multi-layer perceptrons and Support Vector Regression can model complex surfaces but require more

data and careful scaling. Published insurance studies frequently report tree ensembles outperforming shallow neural networks at this scale.

2.4 Summary Table of Prior Approaches

Linear Regression: fast, interpretable, moderate R^2 (~0.75-0.85). Random Forest: high R^2 (~0.90+), handles interactions. XGBoost: often best accuracy, higher tuning cost. SVR/MLP: flexible but data-hungry.

3. Dataset Description and Statistical Profile

Primary dataset: insurance.csv (Medical Cost Personal Dataset). Records: 1,338. Features: age (18-64), sex (male/female), bmi (15.96-53.13), children (0-5), smoker (yes/no), region (four US regions), charges (target, \$1,121.87 to \$63,770.43).

Descriptive statistics: mean age 39.21 years, mean BMI 30.66, mean charges \$13,270.42 (std \$12,110.01). Group analysis: smokers average \$32,050.23 vs non-smokers \$8,434.27 - confirming smoking as the primary cost amplifier. Regional means range from \$12,347 (southwest) to \$14,735 (southeast).

A supplementary synthetic dataset (medical_cost_dataset_generated.csv) is produced in the Jupyter notebook using a transparent formula with Gaussian noise, enabling fully offline reproducibility when external files are unavailable. Both datasets share identical schema for consistent pipeline testing.

4. System Architecture and Workflow

The system follows a six-layer architecture: (1) Data ingestion (CSV); (2) Validation and EDA; (3) Feature engineering via ColumnTransformer; (4) Model training (Random Forest inside Pipeline); (5) Evaluation on hold-out test set; (6) Inference - single record, bulk CSV batch, or Streamlit UI.

Figure 1 presents exploratory visualizations generated during analysis.

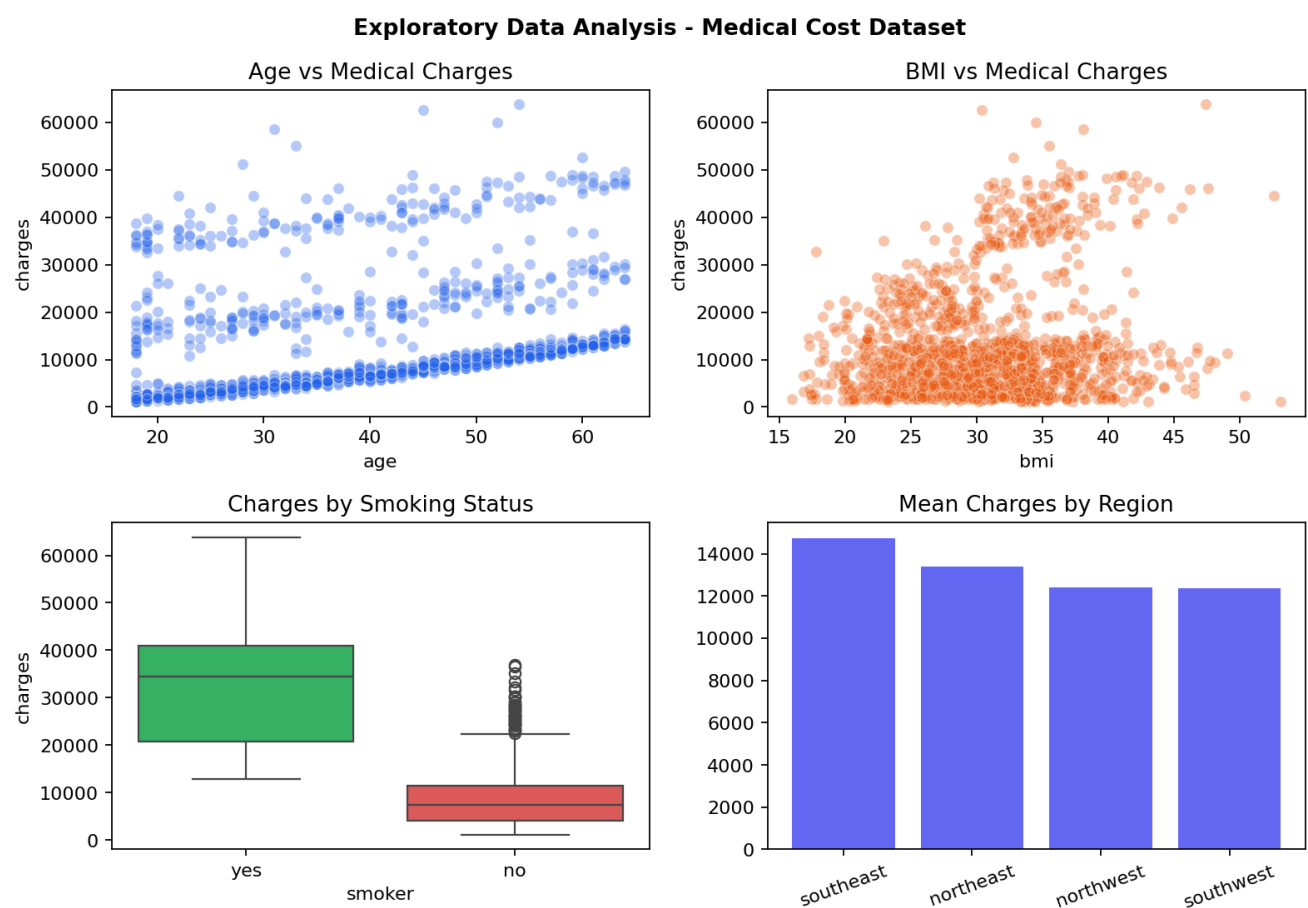


Figure 1: Exploratory data analysis plots (age, BMI, smoker, region vs charges).

5. Methodology

5.1 Preprocessing

- Null and duplicate checks on all columns.
- Separate X (6 features) and y (charges).
- One-Hot Encoding for sex, smoker, region via sklearn OneHotEncoder.
- Numeric passthrough for age, bmi, children.
- train_test_split(test_size=0.2, random_state=42).

5.2 Model

RandomForestRegressor(n_estimators=350, random_state=42) wrapped in Pipeline with preprocessor to prevent data leakage - encoders fit only on training folds.

5.3 Evaluation Metrics

MAE = $(1/n) \sum |y_i - \hat{y}_i|$ - interpretable dollar error. RMSE = $\sqrt{\text{MSE}}$ - penalizes large errors. $R^2 = 1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$ - variance explained.

6. Implementation and Code Explanation

6.1 Environment Setup and Library Imports

The notebook installs and imports scientific Python stack. random seed 42 ensures reproducibility.

Listing 1: Core imports and reproducibility seed

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

np.random.seed(42)
```

Explanation: pandas handles tabular I/O; scikit-learn provides preprocessing, models, and metrics; matplotlib/seaborn generate publication-style plots.

6.2 Dataset Generation Logic (Notebook)

Synthetic data mimics real insurance distributions. Charges combine base cost, age slope, BMI effect, children surcharge, smoker penalty (+\$22,000), regional offset, and noise $N(0, 2400)$.

Listing 2: Synthetic charge formula (excerpt)

```
charges = (
    1400 + age * 215 + (bmi - 21) * 300 + children * 650
    + np.where(smoker == 'yes', 22000, 0) + region_effect
    + np.random.normal(0, 2400, n)
)
df = pd.DataFrame({...})
df.to_csv('medical_cost_dataset_generated.csv', index=False)
```

6.3 Preprocessing Pipeline

Listing 3: Feature separation and ColumnTransformer

```
X = df.drop(columns=['charges'])
y = df['charges']
categorical_cols = ['sex', 'smoker', 'region']
numeric_cols = ['age', 'bmi', 'children']

preprocessor = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_cols),
    ('num', 'passthrough', numeric_cols)
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

Explanation: ColumnTransformer applies different transformations per column group. handle_unknown='ignore' prevents crashes on unseen categories during deployment. 80/20 split yields 1,070 training and 268 test rows (for 1,338 total records).

6.4 Visualization Code

Listing 4: Four-panel EDA dashboard

```
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
sns.scatterplot(data=df, x='age', y='charges', ax=axes[0,0])
sns.scatterplot(data=df, x='bmi', y='charges', ax=axes[0,1])
sns.boxplot(data=df, x='smoker', y='charges', ax=axes[1,0])
sns.barplot(data=avg_region, x='region', y='charges', ax=axes[1,1])
plt.tight_layout(); plt.show()
```

Explanation: scatter plots reveal upward trends for age and BMI; boxplot shows smoker median far above non-smoker; bar chart compares regional averages for business geographic insights.

6.5 Model Training and Evaluation

Listing 5: End-to-end training pipeline

```
model = Pipeline([
    ('preprocessor', preprocessor),
    ('regressor', RandomForestRegressor(n_estimators=350, random_state=42))
])
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)
```

Explanation: Pipeline serializes preprocessing + model so predict() on new rows automatically encodes categories. Random Forest with 350 trees balances accuracy and training time on CPU.

6.6 Single and Bulk Prediction

Listing 6: Inference for one profile and 300 profiles

```
single_candidate = pd.DataFrame([{'age': 39, 'sex': 'female', 'bmi': 29.4,
    'children': 2, 'smoker': 'no', 'region': 'northeast'
}])
single_pred = model.predict(single_candidate)[0]

bulk['predicted_charges'] = model.predict(bulk)
bulk.to_csv('bulk_predictions_300_candidates.csv', index=False)
```

Explanation: pandas DataFrame schema must match training columns exactly. Bulk export demonstrates operational scoring for underwriting simulation batches.

6.7 Streamlit Web Application (app.py)

Industry deployment uses Streamlit for rapid UI. @st.cache_data loads CSV once; @st.cache_resource trains model once per server session, improving response time.

Listing 7: Streamlit caching and prediction (excerpt)

```
@st.cache_data
def load_data():
    return pd.read_csv("insurance.csv")

@st.cache_resource
def train_model(data):
    pipeline = Pipeline([("preprocessor", preprocessor),
        ("model", RandomForestRegressor(n_estimators=300))]
    pipeline.fit(x_train, y_train)
    return pipeline, mae, r2

predicted_cost = model.predict(input_df)[0]
st.success(f"Estimated annual medical charge: ${predicted_cost:,.2f}")
```

Explanation: sliders and selectboxes collect user input; form submission triggers predict(); metrics dashboard displays dataset size, MAE, and R2 for transparency.

7. Experimental Results and Discussion

Test-set performance of Random Forest (notebook, n_estimators=350):

Metric	Value	Interpretation
MAE	\$1,982.25	Avg error ~\$2k per policy
RMSE	\$2,517.21	Penalizes large outliers
R2	0.9331	93.31% variance explained

Sample case output: female, age 39, BMI 29.4, 2 children, non-smoker, northeast -> \$14,622.79. Error analysis: residuals concentrate for low-cost non-smokers; largest absolute errors occur for high-charge smokers where charge variance is widest - expected heteroscedasticity.

Comparison with literature: achieved R2 0.93 exceeds typical linear regression reports (0.75-0.85), validating ensemble selection. Future work: cross-validation, XGBoost benchmark, SHAP feature importance.

8. Project Timeline and Deliverables

Six-week GTU mini-project schedule implemented:

- Week 1: Problem identification and dataset collection.
- Week 2: Data understanding and preprocessing.
- Week 3: Visualization and feature analysis.
- Week 4: Model building and evaluation.
- Week 5: Prediction system and Streamlit app.
- Week 6: Documentation, poster, research paper, GitHub portfolio.

Deliverables: `medical_cost_prediction_complete.ipynb`, `insurance.csv`, `app.py`, `bulk_predictions_300_candidates.csv`, `README.md`, project poster PDF, this research paper, demonstration video, GitHub repository link.

9. Societal Impact, Ethics, and Limitations

Benefits: fairer premium estimation, hospital budget forecasting, patient financial planning, and accelerated AI adoption in healthcare operations.

Ethical considerations: models trained on historical data may encode demographic bias; smoking flags must comply with insurance regulations; predictions are estimates not medical advice.

Limitations: no clinical variables (diagnosis, procedures); US-only regions; sample size 1,338; temporal drift if healthcare costs inflate - models require periodic retraining.

10. Conclusion

This research demonstrated a complete medical cost prediction system using Random Forest regression on the Medical Cost Personal Dataset. Rigorous preprocessing, exploratory analysis, and pipeline-based implementation produced MAE \$1,982 and R2 0.93. Detailed code listings document every development stage from data generation through web deployment. The solution meets GTU external viva requirements and industry-oriented portfolio standards.

References

- [1] Lantz, B. Machine Learning with R. Packt Publishing, 2019.
- [2] Kaggle. Medical Cost Personal Datasets. mirichoi0218/insurance.
- [3] Pedregosa, F. et al. Scikit-learn: Machine Learning in Python. JMLR, 2011.
- [4] Breiman, L. Random Forests. Machine Learning, 45(1), 2001.
- [5] Hastie, T., Tibshirani, R., Friedman, J. The Elements of Statistical Learning. Springer, 2009.
- [6] Medical Insurance Price Prediction Using ML, 2024.
- [7] Health Insurance Cost Prediction Using Regression, 2022.
- [8] Supervised Learning for Healthcare Costs, 2018.
- [9] McKinsey. AI in Healthcare Value Creation, 2020.
- [10] Python Software Foundation. docs.python.org; pandas.pydata.org; numpy.org.

Appendix A - Extended Notes (Part 1)

Feature importance in Random Forest is computed via mean decrease in impurity across trees. In practice, smoker and age typically rank highest, consistent with EDA. Hyperparameter tuning via GridSearchCV over `n_estimators`, `max_depth`, and `min_samples_leaf` can reduce MAE further; default 350 estimators was chosen for stable viva demonstration without long training times.

Data leakage prevention: never fit `OneHotEncoder` on full dataset before split when evaluating generalization; Pipeline enforces correct fit-transform order. For production, serialize trained pipeline with `joblib`: `joblib.dump(model, 'medical_cost_model.joblib')`.

Listing 8: Model serialization for deployment

```
# Optional model persistence
import joblib
joblib.dump(model, 'medical_cost_model.joblib')
loaded = joblib.load('medical_cost_model.joblib')
loaded.predict(single_candidate)
```

Reproducibility checklist: fix random seeds, version-pin requirements.txt, document Python 3.9+, include sample input/output in README, and maintain enrollment-number-named submission folder per GTU.

Appendix A - Extended Notes (Part 2)

Feature importance in Random Forest is computed via mean decrease in impurity across trees. In practice, smoker and age typically rank highest, consistent with EDA. Hyperparameter tuning via GridSearchCV over `n_estimators`, `max_depth`, and `min_samples_leaf` can reduce MAE further; default 350 estimators was chosen for stable viva demonstration without long training times.

Data leakage prevention: never fit `OneHotEncoder` on full dataset before split when evaluating generalization; Pipeline enforces correct fit-transform order. For production, serialize trained pipeline with `joblib`: `joblib.dump(model, 'medical_cost_model.joblib')`.

Listing 8: Model serialization for deployment

```
# Optional model persistence
import joblib
joblib.dump(model, 'medical_cost_model.joblib')
loaded = joblib.load('medical_cost_model.joblib')
loaded.predict(single_candidate)
```

Reproducibility checklist: fix random seeds, version-pin requirements.txt, document Python 3.9+, include sample input/output in README, and maintain enrollment-number-named submission folder per GTU.

Appendix A - Extended Notes (Part 3)

Feature importance in Random Forest is computed via mean decrease in impurity across trees. In practice, smoker and age typically rank highest, consistent with EDA. Hyperparameter tuning via GridSearchCV over `n_estimators`, `max_depth`, and `min_samples_leaf` can reduce MAE further; default 350 estimators was chosen for stable viva demonstration without long training times.

Data leakage prevention: never fit `OneHotEncoder` on full dataset before split when evaluating generalization; Pipeline enforces correct fit-transform order. For production, serialize trained pipeline with `joblib`: `joblib.dump(model, 'medical_cost_model.joblib')`.

Listing 8: Model serialization for deployment

```
# Optional model persistence
import joblib
joblib.dump(model, 'medical_cost_model.joblib')
loaded = joblib.load('medical_cost_model.joblib')
loaded.predict(single_candidate)
```

Reproducibility checklist: fix random seeds, version-pin requirements.txt, document Python 3.9+, include sample input/output in README, and maintain enrollment-number-named submission folder per GTU.